

Legality of Open Source Libraries

Assignment #6

Software Systems Architecture

1MEIC02 T3

Gonçalo Gonçalves da Costa Pinto
José Diogo Alves Granja
Manuel Mo
Pedro Rafael Correia Borges
Ricardo Miguel Yang

1. Context and Problem

PenultimateWidgets lawyers became worried about the license terms of the open source libraries. They wanted to make sure they weren't allowing any viral licenses that force proprietary code to become open-source. For that, they did the initial check of the licenses, but then one critical question arose: **What happens if one of the frameworks changes their license?**

This is not a hypothetical concern. High-profile projects, such as *Redis*, *HashiCorp Terraform* and *Elasticsearch*, have shifted from permissive to restrictive or commercial licenses, which means that companies and developers who use the libraries can find themselves violating the law without doing anything wrong. The challenge is detecting such changes automatically and continuously, rather than relying on periodic manual checks that may miss changes between review cycles.

2. Quality Attributes

To effectively mitigate the risks associated with open-source dependencies, the business requirements must be translated into measurable architectural characteristics:

- **Legal Compliance:** The system must ensure that no direct or transitive dependency carries a license that could expose *PenultimateWidgets* to legal liability, whether because it was never checked during initial integration or because it silently changed after an initial review.
- **Maintainability:** Automating license verification safeguards the long-term sustainability of the system. It prevents a volatile form of technical debt that would require expensive refactoring if a viral license was discovered too late in the development cycle.
- **Auditability:** The system must support the legal team's need to track and verify compliance over time. By automatically generating a baseline of approved dependency licenses, the system creates a reliable architectural audit trail. With that, if a legal dispute arises, *PenultimateWidgets* can easily trace exactly which licenses were approved and active during any specific build or release.

Among these quality attributes, **Legal Compliance** is the most important, as it constitutes the core objective of the proposed solution. Consequently, the fitness function presented in the following section focuses primarily on ensuring legal compliance, while also indirectly reinforcing maintainability and auditability.

3. Fitness Function

3.1 Fitness Function Dimensions

The strategy automates license compliance via a deployment gate governed by two binary thresholds: a **License Violation threshold** and a **License Regression threshold**. The violation metric should equal zero and any deviation immediately fails the pipeline, the regressions are merely notifications that a package's license has changed to one we already allow, nothing breaks. Table 1 outlines the architectural classification and technical reasoning for this implementation.

Dimension	Value	Reason
Atomic vs. Holistic	Atomic	The function evaluates individual library metadata in isolation. Since legal compliance is a binary requirement per dependency, a holistic view of the system's runtime behaviour is unnecessary.
Triggered vs. Continuous	Triggered	It is executed at specific points in the software lifecycle: 1) Upon any change to the dependency manifests (like

		<i>package.json</i> , etc.); 2) Via a scheduled scan (like a cron job) to detect license changes in upstream repositories.
Automated vs. Manual	Automated	The detection of non-compliant licenses is 100% automated. Modifying the allowlist to accept new licenses is the only step that requires manual intervention, and should be performed by a legal expert or senior architect.
Static vs. Dynamic	Static	The check analyzes the contract of the library (metadata and manifest files) at rest. It does not require the code to be executed, as the legal risk is inherent in the inclusion of the package, regardless of its functions being invoked or not during runtime.

Table 1: Classification of the architectural fitness function across the four core dimensions

3.2 Implementation

The programmatic implementation demonstrated in Figure 1 establishes a binary compliance gate within the continuous integration pipeline. The script evaluates current project dependencies by cross-referencing package metadata against an immutable authorization file and a historical state registry.



```

1 # CI step: run after dependency install
2 ALLOWLIST = load_json('.license-allowlist.json')
3 deps      = extract_all_licenses('package-lock.json')
4
5 baseline  = load_json('.license-baseline.json') # committed snapshot
6 violations, regressions = [], []
7
8 for pkg, spdx in deps:
9     if spdx not in ALLOWLIST:
10         violations.append((pkg, spdx))
11     elif baseline.get(pkg) != spdx:          # license changed
12         regressions.append((pkg, baseline[pkg], spdx))
13
14 if violations:
15     report(violations)
16     exit(1)                                # fail the pipeline
17
18 if regressions:
19     report(regressions)
20
21 save_json(deps, '.license-baseline.json')   # update baseline on success

```

Figure 1: Pseudo-code of the fitness function

In our code, all dependencies are extracted from files such as *package-lock.json*. For each dependency, we have the package name and its SPDX (Software Data Package Exchange) license identifier. The first check determines whether the license is allowed by verifying it against the allowlist. If the license is in the allowlist, the code then checks whether the package's license has changed by comparing it to a baseline snapshot that records each package and its associated license. After all dependencies have been checked, conditional logic reports any violations and stops the pipeline. If no violations are found, regressions are also reported. If everything is correct the baseline then saves a new snapshot with the package and SPDX.